Unit 3 – Big Data Study Notes

Spark MLlib | PySpark | Spark Structured Streaming

> **Syllabus:** Spark MLlib – PySpark – Spark Structured Streaming – Basic Concepts, Programming Model, APIs using Streaming DataFrames, Creating Streaming DataFrames, Operations on Streaming DataFrames

---

1. Spark MLlib

Key Concepts

MLlib is Apache Spark's built-in scalable machine learning library. It provides common ML algorithms and utilities that run in a distributed manner across a cluster, making it suitable for large-scale data.

Why MLlib?

- Traditional ML libraries (like scikit-learn) run on a single machine and cannot handle big data.
- MLlib distributes computation across nodes, so training on terabytes of data becomes feasible.
- It integrates natively with Spark's data processing pipeline.

What MLlib Provides

MLlib offers two main APIs:

- **RDD-based API (spark.mllib)** — the older, lower-level API built on Resilient Distributed Datasets.
- **DataFrame-based API (spark.ml)** — the modern, recommended API built on DataFrames. Also called the **Spark ML Pipeline API**.

> **Exam Tip:** The modern API is `spark.ml`. MLlib (`spark.mllib`) is the legacy one.

Core Components of MLlib

Component	Description
Algorithms	Classification, Regression, Clustering, Collaborative Filtering
Featurization	Feature extraction, transformation, selection
Pipelines	Tools to construct, evaluate, and tune ML workflows
Persistence	Save and load models and pipelines
Utilities	Linear algebra, statistics, data handling

ML Pipeline Concept

A **Pipeline** in MLlib is a sequence of stages where each stage is either a **Transformer** or an **Estimator**.

- **Transformer** — Takes a DataFrame as input and outputs a new DataFrame (e.g., adds a feature column).
- **Estimator** — Fits on training data and produces a Transformer (i.e., a trained model). Example: `LogisticRegression` is an Estimator; after `.fit()`, it becomes a Model (Transformer).

Pipeline Stages (Typical Flow):

```
Raw Data → Tokenizer → HashingTF → LogisticRegression → Prediction
```

Each arrow represents a stage in the pipeline.

Important MLlib Algorithms

Classification:

- Logistic Regression
- Decision Tree
- Random Forest
- Naive Bayes

Regression:

- Linear Regression
- Decision Tree Regressor

Clustering:

- K-Means
- Bisecting K-Means

Recommendation:

- ALS (Alternating Least Squares) – used in collaborative filtering

Key Classes

- `Pipeline` — chains multiple stages
- `PipelineModel` — fitted pipeline
- `VectorAssembler` — combines feature columns into a single feature vector
- `StringIndexer` — converts string labels to numeric indices
- `CrossValidator` — hyperparameter tuning using cross-validation

Example / Use Case

Real-World Use Case: Netflix-style movie recommendation using ALS, or spam detection using Logistic Regression on email text features.

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import VectorAssembler
```

```
from pyspark.ml import Pipeline

assembler = VectorAssembler(inputCols=["age", "income"],
outputCol="features")
lr = LogisticRegression(featuresCol="features", labelCol="label")

pipeline = Pipeline(stages=[assembler, lr])
model = pipeline.fit(training_df)
predictions = model.transform(test_df)
```

Important Points

- MLlib works best with the **DataFrame API** (spark.ml).
- Feature vectors must be assembled into a **single vector column** before passing to an algorithm.
- Models can be saved and reloaded using `.save()` and `PipelineModel.load()`.
- MLlib supports **cross-validation** and **train-validation split** for model tuning.

---

2. PySpark

Key Concepts

PySpark is the Python API for Apache Spark. It allows Python developers to write Spark applications using Python syntax while leveraging Spark's distributed computing engine under the hood.

Why PySpark?

- Python is popular for data science; PySpark bridges Python with Spark's power.
- Enables processing of large datasets using familiar Python code.
- Supports Spark SQL, DataFrames, MLlib, and Streaming.

SparkSession — The Entry Point

`SparkSession` is the unified entry point for all Spark functionality. Every PySpark application starts by creating a `SparkSession`.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .master("local[*]") \
    .getOrCreate()
```

- `appName` — name of the application
- `master` — where to run (`local` for local machine, `yarn` for cluster)
- `getOrCreate()` — returns existing session or creates a new one

Core Data Abstractions in PySpark

Abstraction	Description
RDD (Resilient Distributed Dataset)	Low-level, fault-tolerant, distributed collection of objects
DataFrame	Distributed table with named columns; like a SQL table
Dataset	Strongly-typed version of DataFrame (not in Python)

> **Exam Tip:** In Python (PySpark), we primarily use **DataFrames**, not Datasets (Datasets are JVM-only).

RDD — Resilient Distributed Dataset

- **Resilient** — fault-tolerant; can recover from node failures.
- **Distributed** — data is split across multiple nodes.
- **Dataset** — a collection of data items.

Creating RDDs:

```
rdd = spark.sparkContext.parallelize([1, 2, 3, 4, 5])
rdd2 = spark.sparkContext.textFile("hdfs://path/to/file.txt")
```

RDD Operations:

- **Transformations** (lazy, return new RDD): `map()`, `filter()`, `flatMap()`, `reduceByKey()`
- **Actions** (trigger execution): `collect()`, `count()`, `take()`, `saveAsTextFile()`

DataFrame in PySpark

DataFrames are the primary way to work with structured data in PySpark. They support SQL-like operations and are optimized by Spark's **Catalyst optimizer**.

Creating DataFrames:

```
# From a list
df = spark.createDataFrame([(1, "Alice"), (2, "Bob")], ["id", "name"])

# From a CSV file
df = spark.read.csv("data.csv", header=True, inferSchema=True)

# From JSON
df = spark.read.json("data.json")
```

Common DataFrame Operations:

```
df.show()               # Display rows
df.printSchema()        # Show schema
df.select("name")       # Select columns
```

```
df.filter(df.age > 21)  # Filter rows
df.groupBy("dept").count()  # Group and aggregate
df.orderBy("age")       # Sort
df.withColumn("salary_k", df.salary / 1000)  # Add new column
df.drop("column_name")  # Drop a column
```

Spark SQL

PySpark supports running SQL queries directly on DataFrames by registering them as **temporary views**.

```
df.createOrReplaceTempView("employees")
result = spark.sql("SELECT name, salary FROM employees WHERE salary >
50000")
result.show()
```

Lazy Evaluation

Spark uses **lazy evaluation** — transformations are not executed immediately. They build up a **DAG (Directed Acyclic Graph)** of operations. Execution only begins when an **action** is called.

- This allows Spark to **optimize** the entire computation plan before running it.

Important Points

- `SparkContext` is the old entry point (used with RDDs). `SparkSession` is the modern entry point.
- DataFrames are **immutable** — operations return new DataFrames.
- Spark processes data in **partitions** distributed across the cluster.
- PySpark DataFrames use **columnar storage** format internally for efficiency.
- Always call `.stop()` on `SparkSession` at the end of a program to release resources.

---

3. Spark Structured Streaming

Key Concepts

Spark Structured Streaming is a scalable and fault-tolerant stream processing engine built on top of the Spark SQL engine. It allows you to write streaming queries the same way you write batch queries on static DataFrames — the engine handles the continuous processing.

Core Idea — Treat a Stream as an Unbounded Table:

> Structured Streaming treats a live data stream as a table that keeps growing continuously. You query it like a regular table, and Spark continuously appends new results as new data arrives.

Why Structured Streaming?

- Unifies batch and stream processing with the same API.
- Provides end-to-end **exactly-once** fault tolerance.
- Supports event-time processing and late data handling.

- Simpler programming model compared to lower-level streaming (Spark Streaming / DStream API).

Basic Concepts

Stream vs Batch

Feature	Batch Processing	Stream Processing
Data	Finite, stored	Infinite, live
Latency	High (minutes/hours)	Low (milliseconds/seconds)
Use Case	Reports, ETL	Real-time alerts, dashboards

Micro-Batch Processing

By default, Structured Streaming runs in **micro-batch** mode. It collects incoming data into small batches at fixed intervals (trigger intervals) and processes each batch. This gives low latency while maintaining the simplicity of batch logic.

There is also a **Continuous Processing** mode for sub-millisecond latency (experimental).

Event Time vs Processing Time

- **Event Time** — the time when the event actually occurred (embedded in the data).
- **Processing Time** — the time when Spark receives and processes the event.
- Structured Streaming supports event-time processing and can handle **late-arriving data** using **watermarks**.

Programming Model

The programming model for Structured Streaming is simple and mirrors the static DataFrame API:

1. **Define input** — create a streaming DataFrame from a source.
2. **Apply transformations** — filter, aggregate, join, etc.
3. **Define output** — write results to a sink with a specified output mode.
4. **Start the query** — call `.start()` to begin streaming.

Conceptual flow:

```
Source (Kafka / Socket / File)
    → Streaming DataFrame
        → Transformations
            → Sink (Console / File / Kafka / Memory)
```

Output Modes

Output mode defines **what data gets written to the sink** after each trigger:

Output Mode	Description	When to Use
Append	Only newly added rows are written	No aggregations; default for simple streams
Complete	The entire result table is written each time	Aggregations (e.g., counts)
Update	Only rows that have changed since last trigger	Aggregations with updates

> **Exam Tip:** `Append` is the default. `Complete` is used when the full aggregated result is needed. `Update` is the most efficient for aggregations.

Triggers

A **trigger** defines how often a streaming query checks for new data and processes it.

Trigger Type	Description
Default (unspecified)	Runs next micro-batch as soon as previous one finishes
Fixed Interval	Process data at a fixed time interval (`processingTime="10 seconds"`)
One-time	Processes all available data and stops
Continuous	Experimental; very low latency (~1ms)

APIs using Streaming DataFrames

Structured Streaming uses the same **DataFrame/Dataset API** as static Spark. The key difference is in the **source** (streaming source instead of a file) and the **sink** (write stream instead of write).

Key API methods for streaming:

Method	Description
`spark.readStream`	Entry point for creating a streaming DataFrame
`df.isStreaming`	Returns `True` if the DataFrame is streaming
`df.writeStream`	Entry point for writing streaming output
`.format()`	Specifies source/sink format (kafka, socket, console, parquet, etc.)
`.option()`	Sets source/sink specific options
`.outputMode()`	Sets how results are output (append/complete/update)
`.trigger()`	Sets how often to process data
`.start()`	Starts the streaming query
`.awaitTermination()`	Blocks until the query is stopped

Creating Streaming DataFrames

A **Streaming DataFrame** is created using `spark.readStream` instead of `spark.read`.

Source Types

Source	Description
Socket	Reads text from a TCP socket (for testing only)
File	Watches a directory for new files (CSV, JSON, Parquet, etc.)
Kafka	Reads messages from Apache Kafka topics
Rate	Generates rows at a fixed rate (for benchmarking/testing)

Example 1: Socket Source (Testing)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("StreamingExample").getOrCreate()

# Create a streaming DataFrame from a socket
lines = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()

lines.printSchema()
# lines.isStreaming → True
```

> Run `nc -lk 9999` in a terminal to simulate a socket server.

Example 2: File Source (CSV Directory)

```
schema = "name STRING, age INT, salary DOUBLE"

streaming_df = spark.readStream \
    .format("csv") \
    .option("header", True) \
    .schema(schema) \
    .load("/path/to/input/directory/")
```

Spark watches the directory and processes any new files dropped into it.

Example 3: Kafka Source

```
kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "my-topic") \
    .load()

# Kafka gives key and value as binary; cast to string
messages = kafka_df.selectExpr("CAST(key AS STRING)", "CAST(value AS
STRING)")
```

> **Exam Tip:** Kafka is the most important real-world streaming source. Kafka messages are delivered as key-value pairs in binary format.

Operations on Streaming DataFrames

Once a streaming DataFrame is created, you can apply **most of the same operations** as on static DataFrames. However, some operations have restrictions.

Supported Operations

Selection and Filtering:

```
# Word count from socket stream
from pyspark.sql.functions import explode, split

words = lines.select(
    explode(split(lines.value, " ")).alias("word")
)

word_counts = words.groupBy("word").count()
```

Aggregations:

```
# Count of events per category
agg_df = streaming_df.groupBy("category").count()
```

Joins:

- Stream-to-static DataFrame joins are supported.
- Stream-to-stream joins are supported with watermarks.

```
# Stream joined with a static lookup table
enriched = streaming_df.join(static_lookup_df, "user_id")
```

Windowed Aggregations (Event Time):

Windowed operations group events into time windows for aggregation.

```
from pyspark.sql.functions import window

windowed_counts = words \
    .groupBy(window(words.timestamp, "10 minutes", "5 minutes"),
words.word) \
    .count()
```

- `"10 minutes"` — window duration
- `"5 minutes"` — slide interval (how often a new window starts)

Watermarking (Late Data Handling):

A **watermark** tells Spark how long to wait for late-arriving data before closing a window.

```
words.withWatermark("timestamp", "10 minutes") \
     .groupBy(window(words.timestamp, "10 minutes"), words.word) \
     .count()
```

> **Exam Tip:** Watermark = maximum tolerated delay for late data. Events arriving later than the watermark threshold are dropped.

Writing the Output (Sink)

After transformations, write the streaming result using `writeStream`:

```
query = word_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .trigger(processingTime="5 seconds") \
    .start()

query.awaitTermination()
```

Common Sink Formats:

Sink	Description
`console`	Prints output to terminal (for testing)
`memory`	Stores output in an in-memory table (for debugging)
`parquet` / `csv` / `json`	Writes to files in a directory

Sink	Description
`kafka`	Publishes results back to a Kafka topic
`foreach` / `foreachBatch`	Custom output logic per row or per batch

`foreachBatch` — Custom Sink

`foreachBatch` lets you apply any batch DataFrame operation to each micro-batch, giving maximum flexibility:

```
def process_batch(batch_df, batch_id):
    batch_df.write.mode("append").parquet("/output/path")

query = streaming_df.writeStream \
    .foreachBatch(process_batch) \
    .start()
```

Unsupported Operations on Streaming DataFrames

Some operations that work on static DataFrames are **not supported** on streaming DataFrames:

- Multiple aggregations chained (unless using `foreachBatch`)
- `limit()` and `take()`
- `distinct()` without aggregation
- Sorting without aggregation (except in complete output mode)
- `count()` alone (must use `groupBy().count()`)

Fault Tolerance and Checkpointing

Structured Streaming provides **end-to-end exactly-once** guarantees through:

- **Checkpointing** — saves the progress of each query (offsets processed) to a reliable storage (like HDFS or S3).
- **Idempotent sinks** — writing the same data twice does not cause duplication.

```
query = word_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .option("checkpointLocation", "/checkpoint/path") \
    .start()
```

> **Exam Tip:** Always set `checkpointLocation` for production streaming queries. Without it, the stream cannot recover from failures.

End-to-End Example — Word Count from Socket

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split

spark = SparkSession.builder.appName("WordCount").getOrCreate()

# 1. Create streaming DataFrame
lines = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()

# 2. Apply transformations
words = lines.select(explode(split(lines.value, " ")).alias("word"))
word_counts = words.groupBy("word").count()

# 3. Write output
query = word_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

query.awaitTermination()
```

Real-World Use Cases

Use Case	Streaming Source	What is Streamed
Real-time fraud detection	Kafka	Bank transaction events
Live dashboard / monitoring	Kafka	Server logs, metrics
IoT sensor data processing	Kafka / File	Temperature, pressure readings
Social media trending topics	Twitter API → Kafka	Tweets
E-commerce real-time analytics	Kafka	Click-stream, purchase events

---

Quick Revision Summary

MLlib

- Spark's built-in ML library; use `spark.ml` (DataFrame API), not `spark.mllib` (legacy).
- Key concepts: **Pipeline**, **Transformer**, **Estimator**, **VectorAssembler**.
- Supports classification, regression, clustering, and recommendation (ALS).

PySpark

- Python API for Spark; entry point is `SparkSession`.
- Key abstractions: **RDD** (distributed collection), **DataFrame** (structured table).
- Transformations are **lazy**; actions trigger execution.

- Supports Spark SQL via temporary views.

Spark Structured Streaming

- Treats a live stream as an **unbounded table**.
- Uses `spark.readStream` to create a Streaming DataFrame.
- Runs in **micro-batch** mode by default.
- Output modes: **Append**, **Complete**, **Update**.
- Sources: Socket, File, Kafka, Rate.
- Sinks: Console, File, Kafka, Memory, foreachBatch.
- **Watermarking** handles late-arriving data.
- **Checkpointing** ensures fault tolerance and exactly-once semantics.
- Call `.start()` to begin and `.awaitTermination()` to keep the query running.

End of Unit 3 Notes